RAG (Retrieval-Augmented Generation) – Interview Questions with Answers

1. What is Retrieval-Augmented Generation (RAG)?

RAG is an AI architecture that enhances the performance of language models by combining them with external data retrieval. Instead of relying solely on the LLM's training data, it first retrieves relevant documents from a knowledge base, then uses that information to generate a more accurate, up-to-date response.

2. How does RAG improve upon a vanilla LLM like GPT-4?

Vanilla LLMs are limited by their training data cutoff and token constraints. RAG dynamically fetches relevant documents, allowing responses based on current or external data. This improves accuracy, reduces hallucinations, and extends the LLM's effective knowledge base.

3. What are the main components of a RAG pipeline?

- **Retriever**: Retrieves relevant documents using semantic search.
- **Embedder**: Converts documents and queries into vector embeddings.
- **Vector Store**: Stores embeddings and supports similarity search (e.g., FAISS, Pinecone).
- **Generator**: The LLM that uses the retrieved context to generate responses.

4. Compare RAG vs fine-tuning — when would you choose one over the other?

- **RAG**: Ideal for dynamic or large-scale knowledge that changes often. No model retraining needed.
- **Fine-tuning**: Better when you want the model to internalize domain-specific language or style, and when the knowledge base is stable.

5. Explain the step-by-step flow of a RAG pipeline.

1. User inputs a query.
2. The query is embedded using an embedding model.
3. Vector similarity search retrieves top relevant documents.
4. Retrieved documents are combined with the original query.
5. LLM uses this context to generate a response.

6. How do you retrieve relevant documents in a RAG setup?

By converting the input query into a vector and performing a similarity search against a vector store of pre-embedded documents.

7. What vector store have you used with RAG?

Common options include: - **FAISS** (local, fast, open-source) - **Pinecone** (cloud-based, scalable) - **ChromaDB**, **Weaviate**, **Milvus**, **Redis** (with vector support)

8. What is the role of embedding models in RAG?

They transform text into dense vector representations. This allows the system to compute semantic similarity between queries and documents, enabling effective retrieval.

9. How would you implement a simple RAG pipeline using Python?

Use HuggingFace or LangChain: - Load embeddings (e.g., SentenceTransformers) - Index docs in FAISS - Use OpenAI or local LLM with retrieved docs as context

10. Which embedding model would you choose for semantic search? Why?

- `all-MiniLM-L6-v2` for fast and good quality
- `text-embedding-3-small` from OpenAI for better contextual performance
- Use domain-specific models for technical or biomedical data

11. How do you preprocess and chunk documents before indexing?

- Remove stopwords, clean formatting
- Chunk by paragraph or sentence with overlap (e.g., 300 tokens with 50 overlap)
- Ensure semantic coherence in chunks

12. How do you evaluate the performance of a RAG system?

- **Human eval**: Accuracy, completeness
- **BLEU/ROUGE** scores (for known answers)
- **Recall@K**: Relevance of retrieved docs
- **Latency** and **response time**

13. What is a good chunk size for text retrieval in RAG?

Typically 200–500 tokens per chunk, with 20–50 token overlap. This balances context size with retrieval precision.

14. What are the main limitations of RAG?

- Latency due to retrieval step
- Irrelevant or low-quality retrieval hurts output
- Requires well-maintained vector store
- Complex to scale and keep updated

15. How do you reduce hallucination in RAG pipelines?

- Ensure high-quality, relevant retrieval
- Use grounding prompts that refer directly to context
- Filter or rank retrieved docs based on confidence

16. How do you ensure context relevance in multi-turn conversations using RAG?

- Maintain a conversation history buffer
- Summarize past queries and use them in retrieval
- Apply context window sliding techniques

17. How can you use metadata filtering in RAG?

Store document metadata (e.g., tags, author, date) in vector store. Use filters during retrieval to narrow down search scope.

18. Can you combine RAG with structured data (SQL/GraphQL)?

Yes. Retrieve documents via vector store and combine responses with real-time data from structured sources via agents or tools.

19. How do you keep your RAG knowledge base up to date?

- Periodically re-index new documents
- Use a document watcher for real-time updates
- Automate embedding + upsert workflows

20. How would you scale a RAG system for production?

- Use cloud-hosted vector DB (e.g., Pinecone)
- Cache common queries
- Async API calls
- Load balance across GPU instances

LangChain – Interview Questions with Answers

1. What is LangChain and why is it useful?

LangChain is a Python framework that simplifies building applications powered by language models. It abstracts away prompt engineering, chaining, memory, agents, and tool integration.

2. What are the core modules of LangChain?

- **LLMs**: Wrapper for OpenAI, HuggingFace, etc.
- **Chains**: Sequence of operations
- **Agents**: Reasoning and tool usage
- **Memory**: Manages context over conversations
- **Tools**: External APIs, databases, calculators, etc.

3. How does LangChain support building LLM-powered apps?

It provides pre-built components (prompt templates, chains, memory, agents) to assemble apps quickly. It reduces boilerplate and increases modularity.

4. What is a chain in LangChain?

A chain is a sequence of actions or components that process input and generate output using an LLM.

5. Difference: `SimpleSequentialChain` vs `SequentialChain`?

- `SimpleSequentialChain`: Passes the output of one prompt directly into the next
- `SequentialChain`: Supports multiple inputs/outputs with variable names

6. How do you build a custom chain?

By subclassing `Chain` or using `LLMChain`, and composing steps using input/output keys. You can also combine chains using `SequentialChain`.

7. What is an agent in LangChain?

An agent is a component that decides which action (tool or chain) to take next, based on the LLM's reasoning. Useful for dynamic tasks.

8. How do agents differ from chains?

- **Chains** follow a fixed path
- **Agents** decide actions at runtime based on user input and context

9. What is a tool in LangChain?

A callable utility (e.g., search API, calculator, SQL DB) that the agent can use during its reasoning process.

10. Example use case for agents?

- Building a chatbot that can answer FAQs, do web searches, query SQL databases, and perform calculations — all dynamically.

11. What is memory in LangChain?

Memory stores past interactions to maintain context across turns. It can be simple buffer memory or more complex like summary or vector memory.

12. How does `ConversationBufferMemory` work?

It keeps a buffer of the most recent messages, appending them to prompts for context preservation.

13. How do you manage context in multi-turn conversations?

- Use memory (buffer, summary, entity memory)
- Limit history size to stay within token limits
- Use context compression if needed

14. How do you integrate LangChain with FAISS or Pinecone?

- Load embeddings
- Store and retrieve documents
- Pass retrieved context into chains/agents

15. How do you use LangChain with OpenAI or HuggingFace?

- Use `ChatOpenAI` or `HuggingFaceHub` LLM wrappers
- Plug into `LLMChain`, `Agent`, or custom apps

16. Can you use LangChain with local LLMs?

Yes. You can use wrappers for `llama-cpp`, `ollama`, or HuggingFace models running locally.

17. How do you use LangChain with document loaders?

Use `langchain.document_loaders` to load PDFs, text, web pages, then split, embed, and index with vector stores.

18. Build a chatbot using LangChain that answers FAQs?

- Load FAQ documents
- Chunk and embed them
- Use `RetrievalQA` chain with vector store
- Add conversational memory and frontend (e.g., Streamlit)

19. How would you build a voice assistant with LangChain?

- Speech-to-text (e.g., Whisper)
- Process input through agent or chain
- Text-to-speech (e.g., pyttsx3 or gTTS)

20. How do you add prompt templating and output parsing?

Use `PromptTemplate` and `OutputParser` classes to enforce structure in both input and output of the LLM.